Time-Delayed Publicly Verifiable Quantum Computation for Classical Verifiers

Ameer Mohammed*
ameer.mohammed@ku.edu.kw
Kuwait University
Kuwait City, Kuwait

Aydin Abadi*
aydin.abadi@ncl.ac.uk
Newcastle University
Newcastle upon Tyne, United Kingdom

Jaffer Mahdi*
jaffer.mahdi@ku.edu.kw
Kuwait Oil Company
Kuwait City, Kuwait

Abstract

Publicly verifiable delegation is a well-known problem involving a user who wishes to outsource a resource-intensive computational task to a more powerful but potentially untrusted server such that any other party is able to efficiently check the veracity of the computation's result. This problem has been extensively studied in the classical domain where the user and server are both non-quantum machines. However, the problem becomes more challenging when the classical user wants to delegate a quantum circuit to a *single* prover with quantum-computing capabilities. Previous solutions have resorted to using impractical or non-standard cryptographic solutions (e.g. indistinguishability obfuscation) to achieve this requirement. In this work, we relax the requirement to have *time-delayed* publicly verifiable proofs, where the verification key is made known to the public only when the computation (and its proof) are guaranteed to have been completed. We propose a practical non-interactive scheme leveraging commitment schemes and time-lock puzzles, which can be efficiently realized through well-established and standard post-quantum assumptions. The main idea of our technique lies in using time-lock puzzles to compile a 2-round privately verifiable scheme into a non-interactive publicly verifiable scheme with timestamped proofs, outsourcing not only the quantum computation but the puzzle solving as well. Security is proven in the quantum random oracle model with a common reference string (CRS).

CCS Concepts

• **Security and privacy → Cryptography**; *Security services*.

Keywords

Verifiable Computation, Publicly Verifiable, Time-lock Puzzles, Post-Quantum Security

ACM Reference Format:
Ameer Mohammed, Aydin Abadi, and Jaffer Mahdi. 2026. Time-Delayed Publicly Verifiable Quantum Computation for Classical Verifiers. In *Proceedings of Conference on Computer and Communications Security (CCS '26).* ACM, New York, NY, USA, 15 pages. https://doi.org/XXXXXXX.XXXXXXX

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
CCS '26, November 15–19, 2026, The Hague, The Netherlands
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/18/06
https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

An important problem when outsourcing a client's computation to a more capable server is ensuring that the result that is computed and returned by the server is correct and efficiently verifiable by the client. In particular, the verification complexity should be much lower than that required to run the actual computation. This problem has been well studied when the server and client are both non-quantum machines, and, in fact, can be realized through the use of succinct arguments for **NP** [61, 74]. With quantum computing on the horizon and likely to be offered initially as a pay-per-use service by major companies, it is vital to prepare for a scenario where users with classical computers want to delegate complex computations to powerful quantum servers and efficiently verify the results produced by the servers– a task now known in the literature as Classical Verification of Quantum Computation (CVQC).

In the breakthrough work of Mahadev [67], it was demonstrated that such a scheme does indeed exist assuming the post-quantum hardness of Learning With Errors (LWE). This was further extended [28, 11] to add succinctness, in which the verifier's time complexity is $\text{poly}(\log T)$ where T is the time to execute the delegated quantum computation. In addition, while the original protocol consisted of 4 rounds of communication, this was reduced to 2 rounds or less (1 round with setup) while still preserving negligible soundness through parallel repetition [28, 7, 11].

More relevant to this work, some applications demand *public verifiability*, an additional property allowing anyone (not just the delegator) to verify the result of the computation.

The usual desired setting of publicly verifiable non-interactive verifiable computation involves a prover who wishes to publish a single-message proof convincing any verifier that $C(x) = y$ for some program or circuit C over input x. To avoid triviality in the fully classical setting, the proof needs to be relatively "short" such that verification is faster than re-executing $C(x)$ and checking the result, otherwise the verifier has no need to delegate and can instead perform the computation itself. The proof size requirement, while preferred, is not highly prioritized when the prover is quantum, as C is not necessarily efficiently computable by the verifier. The standard go-to approach to solve this problem is by using succinct non-interactive arguments (SNARGs) for **NP** (or even **P**), which can be realized from LWE [29]. However, our delegated computation is quantum, and thus one needs a SNARG for **BQP** for this approach to work. The work of Bartusek and Malavolta [13] provides one means to achieve this goal, and, in fact, they obtain SNARGs for **QMA**, the quantum analogue of **NP**, (and hence **BQP**) which would imply publicly verifiable CVQC. Their results rely on quantum null-iO, which is derived from non-standard cryptographic primitives, e.g.,

VBB obfuscation. In contrast, our goal is to prioritize practicality over theoretical feasibility, and therefore, we wish to avoid such strong assumptions.

Ideally, we would like to base the scheme on *falsifiable assumptions* [79, 39], which are those that can be tested in a security game against an efficient challenger. Examples include one-way functions, trapdoor permutations, DDH, and LWE. Notably, indistinguishability obfuscation is *not* falsifiable [52]. Roughly speaking, such assumptions are often preferred, as they can be efficiently proven/refuted and therefore instill more confidence when used as building blocks for more advanced schemes. The very recent related work of Bartusek et al. [10] comes close to achieving this by showing a publicly verifiable CVQC that is based on LWE but they do so under the classical oracle model [12], which idealizes the notion of obfuscation for classical circuits and can be heuristically instantiated with iO.

1.1 Our Result

We seek to make progress toward addressing the following question:

*Does a publicly verifiable delegation protocol for quantum computation with a classical verifier exist under **falsifiable** assumptions in the random oracle model?*

We answer the question affirmatively with a minor adjustment to the standard publicly verifiable computation setting. Unlike standard publicly verifiable computation, where the verification key is available from the outset, our scheme delays the release of this capability. Specifically, public verifiability is enabled only after a time delay Δ, where $\Delta > O(T)$, and only for proofs generated before time Δ. Until that point in time, the scheme operates in a privately verifiable mode.

Theorem 1.1 (informal). *Under the post-quantum LWE assumption, there exists a non-interactive time-delayed publicly verifiable CVQC protocol for problems in **BQP** with computational soundness in the common reference string (CRS) and quantum random oracle model (QROM).*

A scheme in the common reference string (CRS) model, as opposed to the plain model, would allow all parties to have access to a string that was honestly generated by some trusted setup algorithm. We note that the CRS is not uniformly random but is deliberately structured, so we cannot simply replace the need for a CRS with an output of the random oracle.

Our technique of employing time-lock puzzles to render a CVQC protocol publicly verifiable is also of independent interest. To the best of our knowledge, this is the first instance in which time-lock puzzles are used to transform a designated-verifier verifiable quantum computation scheme into a publicly verifiable one.

We expect that this technique can be generalized to compile any privately verifiable computation scheme into a time-delayed, publicly verifiable one, even in the classical (non-quantum) setting. However, since standard publicly verifiable delegation schemes based on falsifiable assumptions already exist in the classical setting [56, 41], this generalization is more impactful and meaningful in the classical-verifier quantum-prover setting. We expect that the time-delayed publicly verifiable scheme offers greater efficiency in scenarios where lightweight alternatives to SNARGs are preferred.

We implemented an instantiation of our scheme for two computational tasks on AWS Braket, which is a cloud-based quantum simulation platform. We tested our scheme on random quantum circuits and Harrow–Hassidim–Lloyd (HHL) circuits [46, 93], and estimated their concrete running time. The latter was selected as one potential candidate circuit that a client may delegate to take advantage of the quantum speedup and solve linear systems exponentially faster than classical algorithms. In assessing concrete running times for a 16×16 HHL instance, we found that circuit computation by the quantum prover took 90 seconds (which is also the time it should take at least to solve the corresponding puzzle) while puzzle generation on the classical client side required only 0.05 ms.

1.2 Motivation and Applications

Quantum computers promise significant speedups over classical machines for a wide range of tasks. However, due to their high cost and limited availability, they are expected to be initially accessed via cloud-based, pay-per-use services. In such a model, clients (often classical and resource-limited) need to outsource their quantum computations to powerful remote quantum servers. This delegation model introduces the critical challenge of trust:

How can a classical client be sure that the result provided by a remote quantum server is correct?

Verifiable delegation schemes aim to solve this by enabling the client to efficiently verify the correctness of the quantum result without redoing the computation themselves. In many real-world scenarios, it is not enough for only the original client to verify the result of a delegated quantum computation; instead, the result should be publicly verifiable. For instance, public verifiability becomes essential when (a) the client is offline or resource-constrained, or (b) dispute resolution is needed and third parties (e.g., courts, auditors, or regulatory bodies) must confirm correctness. In particular, in commercial settings such as quantum cloud services, public verifiability strengthens accountability and helps prevent spurious disputes: customers and auditors can independently validate reported outputs, and providers are protected against false accusations of misbehavior. More broadly, these deployment requirements (offline or resource-constrained clients, dispute resolution, and external auditability) motivate publicly verifiable CVQC. At the same time, achieving public verifiability is nontrivial because existing CVQC protocols are typically designated-verifier and rely on secret verification keys; Section 1.3 discusses why naive "publish-the-key" strategies fail. Publicly verifiable CVQC is therefore particularly relevant in high-stakes domains where outputs affect consequential decisions and post-hoc auditability is required, for example:

- Drug Discovery: Quantum algorithms like Quantum Approximate Optimization Algorithm and Variational Quantum Eigensolver can simulate complex molecular interactions and predict protein folding patterns more efficiently than classical methods. This not only accelerates the drug discovery process but also deepens our understanding of protein misfolding diseases, including Alzheimer's and Parkinson's [35, 57].
- Genomics and Personalized Medicine: Quantum computing has the potential to transform genomics and personalized medicine by facilitating the analysis of complex genetic interactions on a scale beyond the reach of classical computers [30, 83].

- Portfolio Optimization and Risk Modeling: Quantum algorithms can solve complex optimization problems inherent in portfolio management, enabling better asset allocation and risk assessment [92]. For instance, the Harrow-Hassidim-Lloyd algorithm can be employed for such purposes [45].
- Quantum-Enhanced Classification: Quantum machine learning models such as Quantum Support Vector Machines and Variational Quantum Classifiers can substantially improve data classification [48]. These models have shown promise in fields like fraud detection and medical diagnosis by offering potential computational advantages over classical classifiers [51, 71].

1.3 Naive Approaches

One might wonder why the client cannot use a privately verifiable scheme and publish the secret key in the clear alongside the circuit to enable third-party verification. Publishing the key undermines security: once it becomes public, a malicious prover can forge proofs that pass verification. Even if the client privately shares the key with a select set of verifiers, verification remains non-public, because only those parties can check the result. Worse yet, if any verifier leaks the secret to the quantum server, the server could generate convincing but invalid proofs, completely breaking soundness.

Alternatively, if the client were allowed to send a second message, it could keep the secret verification key private until the server returns the result and proof, and then publish the key so that anyone can verify. In our setting, this "publish-later" strategy is not available: it requires either (i) continued client availability to issue a second release message, or (ii) a trusted escrow that stores the key and releases it at the appropriate time. We assume neither, the client may go offline immediately after its first-round message, and we do not rely on any trusted party for timed key release. Consequently, delayed public verification must embed the key material in the client's initial message in a way that prevents recovery before the prescribed time and lets any party later recover the same key fixed at setup and check its consistency with the public parameters.

1.4 Technical Overview

Before presenting the high-level technical ideas behind our approach, we first highlight the core challenges by examining several strawman solutions which may at first glance appear viable but ultimately fail to address our goals.

1.4.1 Initial Attempts. As previously mentioned, SNARGs for **BQP** are the best candidate for solving this problem. However, their construction currently relies on very strong assumptions [13]. One might consider adopting an approach similar to [28] (in the CRS model) where a fully homomorphic encryption (FHE) of the private verification key is sent by the client to third parties where they will, in turn, run a (publicly executable) FHE evaluation of the verification procedure over the ciphertext to output an encrypted result of the verification. However, the third-party verifier will not be able to decrypt the prover's homomorphically evaluated result, as they do not possess the FHE secret key.

To avoid dealing with encrypted results, a somewhat related attempt is to use functional or even attribute-based encryption (ABE) as proposed in [80] where a delegated Boolean circuit C is represented as a functional secret key sk_C, the input x to the circuit is represented as an attribute on a ciphertext c_x of a randomly encrypted message, and computing the result consists of decrypting c_x using sk_C to get $C(x)$. That said, because C is a quantum circuit, we need ABE for circuits C in **BQP**, which again is only known to be achievable from iO [13]. Alternatively, we can try to let the functional secret key be $sk_{V_{vk}(.)}$ where V is the *classical* verifier circuit with the private verification key hardcoded. For this, we need some form of public-key function-private ABE, which is known to be restricted to a limited class of functions, e.g., point functions or inner products [24].

To circumvent the apparent lack of sufficiently expressive primitives from standard assumptions, we elected to relax the public verifiability requirement to be active only after the computation and its proof are released. This intuition leads us towards using time-based cryptography as a means to delay verifiability for the benefit of basing our scheme on sturdier assumptions.

1.4.2 Our Approach. At a high level, the core idea behind our protocol is to transform an efficient, privately verifiable CVQC scheme into a publicly verifiable one by encapsulating the private verification key within a time-lock puzzle, a cryptographic time capsule. The verification key is released only after the delegated computation is completed. The steps are illustrated in Figure 1, which we describe in more detail here.

The privately verifiable protocol that we start with relies on the measurement protocol of Mahadev [67] which generates a secret trapdoor (of a trapdoor injective claw-free function, TCF) for use during the verification. We require the client, during the setup, to encode the secret trapdoor used for the verification in a time-lock puzzle and publish it as part of the public parameters of the scheme.

The client publishes the public parameters along with the puzzle. The server performs the delegated computation and publishes the result along with a proof asserting the computation has been performed correctly. Later, the server finds the solution to the puzzle and publishes the solution too. The puzzle's difficulty is set such that the server cannot find its solution before performing the computation and generating the proof. Thus, if $C(x)$ takes time T to complete and prove, we set the puzzle to be solved in time $\Delta > \Omega(T)$. Given the computation result, proof, and the trapdoor, any verifier can check the solution's validity. Since the server has already computed the result and published it, the knowledge of the trapdoor that it finds later will not help it to cheat and create a proof for a false statement.

Furthermore, the server cannot wait until after solving the puzzle and then publish a forged proof, because the proof must carry a verifiable timestamp τ, and verification rejects unless τ is smaller than the deadline. The verifiers do not need to monitor the server; they only check the timestamp embedded in the proof against the public deadline.

However, we still need to make sure the verifiers are given the original trapdoor, otherwise the server might be able to provide an alternative trapdoor for an invalid computation result but still convince the verifiers to accept the proof. To satisfy this requirement, we use the following efficient technique, introduced in [3]. When the client generates the time-lock puzzle for the trapdoor, it generates a commitment to this trapdoor and publishes the commitment

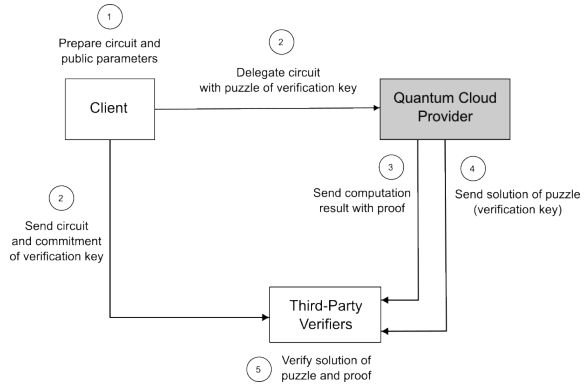


Figure 1: A simplified diagram illustrating the high-level structure of the post-quantum PV-CVQC protocol using time-lock puzzles and commitments.

as part of the public parameters. During the generation of the time-lock puzzle, the client concatenates the plaintext trapdoor with the fixed-length random value used for the commitment and generates the puzzle for this plaintext, i.e., the concatenation. In this setting, when the server solves the puzzle, it parses the solution into two parts: the trapdoor and the randomness (which are the opening of the commitment) and publishes them. Using the opening, a verifier initially checks whether they match the public commitment and if the verification passes, then the verifier uses the trapdoor to check the correctness of the computation's result. To ensure that the time-lock puzzle cannot be solved by the quantum prover faster than intended, it must be based on a post-quantum hardness assumption. Since TCF was shown to be realized under the LWE or Ring-LWE assumptions [22, 23], which is secure against poly-time quantum adversaries, we can rely on the same assumptions to realize a time-lock puzzle primitive as recently shown in [62, 5].
To summarize, informally, the protocol steps are as follows:

(1) The client, who wants to delegate a circuit C on input x, generates the public pk and private sk parameters of the underlying privately verifiable CVQC scheme.
(2) It publishes a time-lock puzzle o and a commitment d of the private trapdoor alongside the circuit to be delegated.
(3) The server runs the CVQC prover's algorithm on the delegated computation $C(x)$. It publishes the computation output along with the associated proof of correctness. Concurrently, the server may start solving the puzzle o.
(4) Once the puzzle solution, representing the now-public trapdoor, is recovered, the server publishes it to all verifiers.
(5) The verifier, now having access to the CVQC trapdoor verification key, can check the validity of the puzzle solution against the commitment d, and if it passes, initiates the CVQC verification algorithm to either accept or reject the proof.

We elaborate on each of these steps and the implementation of each algorithm more formally in Section 4.

2 Related Work

This section covers previous work in verifiable quantum computation and public delegation as well as techniques from timed cryptography that are adjacent to our work. Although there have been several works addressing the verifiable quantum computation problem, we will focus mainly on the setting of a single classical verifier and single quantum prover, the most notable of which are listed in Table 1.

2.1 Verification of Quantum Computation

Approaches used for quantum computation verification can be categorised according to the model of computation that parties are assumed to possess within a particular setting and whether certain computational assumptions are made. If the verifier is imbued with some lightweight quantum capabilities (e.g., measurement), then various solutions exist in this setting [14, 38, 77, 76, 88, 58]. There are also methods in which the verifier is kept classical, but the protocol requires two or more entangled provers [55, 42, 32]. While these protocols can be shown to have information-theoretic security, having multiple entangled and non-communicating provers is often practically prohibitive.

Borrowing from ideas in computational security, a recent promising line of work, initiated by Mahadev [67], deviates from the above approaches by making additional computational assumptions to obtain verifiable delegation in a setting where the verifier is an efficient classical machine interacting with a single (computationally bounded) prover. This was the first such interactive protocol in this setting, verifying the result of a quantum computation using four rounds of communication. This protocol was proven to be sound under the learning with errors (LWE) assumption [85], which is assumed to be intractable for polynomial-time quantum machines. Subsequent works [28, 7] reduced the number of rounds to 2 (or one round with setup) in the quantum random oracle model, and in fact, we use these as a basis for our protocol.

Gheorghiu and Vidick [40] propose a composable protocol that allows a verifier to delegate to a prover the preparation of certain single-qubit quantum states. The composability feature of the protocol allows it to be securely combined with other protocols. The protocol, in a preprocessing phase, requires several rounds of communication that depend on the circuit size. It also relies on LWE. Chung et al. [31] propose a generic compiler that transforms any CVQC protocol to a blind one while preserving the original protocol's round complexity. It relies on quantum fully homomorphic encryption schemes and quantum-secure LWE.

Delegation protocols can also have the property of blindness where the prover is guaranteed to learn nothing about the circuit being delegated (apart from its size). However, impossibility results have been established for *information-theoretically* secure blind delegation of quantum computation to classical verifiers [1].

All of the aforementioned protocols are in the designated verifier model where the verifier needs to possess some secret information for verification; if the prover gains access to this secret then it can convince a verifier about an invalid proof. As a result, they are not publicly verifiable.

Protocol	Round Complexity	Succinct	Blind	Publicly Verifiable	Assumptions	Model
Mahadev [67]	4	No	No	No	LWE	Plain
Gheorghiu and Vidick [40]	$O(\|C\|)$	No	Yes	No	LWE	Plain
Chia et al. [28]	2	Yes	No	No	LWE + iO	QROM+CRS
Alagic et al. [7]	1 (w/setup)	No	No	No	LWE	QROM
Zhang [94]	$O(\lambda)$	Yes ($O(\|C\|)$)	No	No	LWE	QROM
Bartusek and Malavolta [13]	1	Yes	No	Yes	LWE + iO	QROM + CRS
Metger et al. [73] and Gunn et al. [44]	$O(1)$	Yes	No	No	LWE	Plain
Bartusek et al. [12, 10]	1	Yes	Yes	Yes	LWE	Classical Oracle Model
This work	1 (w/setup)	No	No	Yes	SIS/LWE variants	QROM + CRS

Table 1: The existing CVQC schemes in the single verifier, single prover setting and their properties. Succinctness refers to the property that verification time is $\mathrm{poly}(\lambda, \log d(C))$ where λ is the security parameter and $d(C)$ is the depth of the delegated circuit C. Blindness refers to keeping the computation hidden from the prover. All computational assumptions are post-quantum.

2.2 Publicly Verifiable Delegation

Publicly verifiable delegation has long been studied for the setting where the verifier and prover are both classical machines. This is reflected in the long line of work that proposes various schemes under standard [56] and non-standard or idealized (e.g., Random Oracle Model) assumptions [16, 15]. Commonly used methods for achieving publicly verifiable proof systems involve the application of the Fiat-Shamir heuristic [36] to convert a public-coin interactive proof system (e.g. sigma protocols) into a non-interactive one [63, 27, 25] or rely on trusted setup assumptions [43, 17, 81] such as common reference strings (CRS). Such techniques generally give rise to non-interactive zero-knowledge proofs (NIZKs) and succinct non-interactive arguments of knowledge (SNARKs) with the public verifiability property.

In the quantum setting, several constructions of NIZKs for QMA have been proposed under different settings and/or assumptions. The works of [33, 78] construct NIZK for QMA with quantum pre-processing in the designated-verifier model where the proof is classically verifiable but the verifier needs to generate a quantum message as its first message. In addition, several constructions were proposed under the malicious designated verifier (MDV) model [87, 9] where the trusted setup generates a *reusable* common random string but any verifier, given the CRS, needs to generate its own classical public-secret key pair where the secret key is used for its own private (designated) verification. All of the above protocols require some form of secret parameters for verification and are therefore not publicly verifiable.

Previous work on publicly verifiable quantum computation relied on the verifier having some form of quantum capability [49, 65]. In the CVQC setting, a recent line of work [13, 12, 10] focuses on achieving public verifiability based on (post-quantum) indistinguishability obfuscation (iO) for classical circuits or under the classical oracle model, which idealizes obfuscation for classical circuits. Our work aims to eliminate the (strong) iO requirement. While follow-ups to this work [73, 44] did achieve succinct arguments for **QMA**by forgoing iO and relying only on (post-quantum) LWE only, their schemes are interactive with constant round complexity, while we wish to keep the communication non-interactive.

2.3 Time-Based Cryptography

2.3.1 Time-Lock Puzzle. Timothy May [72] initially put forth the idea of sending information to the future. A basic property of a time-lock scheme is that generating a puzzle takes less time than solving it. The scheme that Timothy May proposed used a trusted agent that releases a secret on time for a puzzle to be solved. Since relying on a trusted agent can be a strong assumption, Rivest *et al.* [86] proposed an RSA-based TLP. This scheme does not require a trusted agent, relies on sequential squaring, and is secure against a receiver who may have many computational resources that run in parallel with the goal of finding the solution faster. Since the introduction of the RSA-based TLP, different variants of it have been proposed, including post-quantum TLPs [62, 5] and homomorphic TLPs, e.g., in [70]. Malavolta and Thyagarajan et al. [70] proposed the notion of homomorphic TLPs, which allows an arbitrary function to run over puzzles of different clients before they are solved. The schemes use the RSA-based TLP and fully homomorphic encryption. Furthermore, to achieve efficiency, partially homomorphic TLPs have also been proposed, including those that support homomorphic linear combinations of puzzles [70, 66].

2.3.2 Verifiable Delay Function (VDF). A VDF enables a prover to provide a publicly verifiable proof stating that it has performed a pre-determined number of sequential computations [18, 90, 19, 82]. VDF was first formalized by Boneh *et al* [18]. They proposed several VDF constructions based on SNARKs along with either incrementally verifiable computation or injective polynomials, or based on time-lock puzzles, where the SNARKs-based approaches require a trusted setup. Later, Wesolowski [90] and Pietrzak [82] concurrently improved the previous VDFs from different perspectives and proposed schemes based on sequential squaring. They also support efficient verification. Most VDFs have been built upon TLPs. Nevertheless, the converse is not necessarily the case because VDFs are not suitable to encapsulate an arbitrary private message since they take a public message as input, whereas TLPs have been designed to conceal a private input message.

2.3.3 Timed Commitment. A timed commitment is a commitment scheme that includes an optional forced opening phase, enabling the receiver to recover the committed value without the committer's cooperation—provided the receiver invests sufficient computational effort and time [21, 4, 59]. This property is particularly useful in

scenarios where the sender refuses to voluntarily open the commitment. In our solution, a timed commitment can be employed as a black-box component to unify the time-lock puzzle and commitment schemes, albeit at the expense of efficiency. This is because, in our scheme, the client is assumed to be honest and therefore does not need to prove (e.g., via zero-knowledge proofs or related techniques) that the puzzles have been generated correctly, an assurance that is typically required in timed commitment schemes. Therefore, we adopt an efficient technique proposed in [3], which combines a time-lock puzzle with a commitment scheme, enabling the server to efficiently prove that it has recovered the correct solution originally embedded by the client within the puzzle.

2.3.4 Timed Signatures and Proofs. Researchers have proposed the timed version of digital signatures [2]. They formally defined such a notion in the universally composable (UC) framework [26] and proposed a protocol to realize this concept by mainly relying on a blockchain and standard digital signature scheme. Arun et al. [8] proposed the notion of short-lived zero-knowledge proofs and signatures, which consider the case that after a certain time period the proofs or signatures will be invalid/forgeable. The main idea behind the aforementioned notion is to allow any party to forge any proof by executing a large sequential computation; the proposed schemes primarily rely on VDFs. This differs from our setting where instead of eventual deniability, we are concerned with eventual public verifiability. To enable a smart contract to efficiently verify a proof in a "proofs of data retrievability" system, researchers combined a message authentication code (MAC) with an RSA-based TLP. The MAC verification key is embedded in the TLP and provided to the smart contract after the proof is registered [3].

3 Preliminaries

In this section, we present the background and necessary material for the models, primitives, and protocols used to build our scheme.

3.1 Notation

We say that a function $f : \mathbb{N} \to [0, 1]$ is negligible if, for any polynomial $p(.)$ and sufficiently large $n \in \mathbb{N}$, we have $f(n) < 1/p(n)$. We denote $\mathcal{F}(X, Y)$ to be the set of all functions with domain X and range Y. Given two interactive algorithms P and V, we denote $y \xleftarrow{\$} \langle P(w), V(z)\rangle(x)$ as the random variable representing the interaction between P (with private input w) and V (with private input z) on common input x where y is V's local output.

We say that a classical algorithm $A(x)$ is efficient or probabilistic polynomial time (PPT) if it runs in time poly($|x|$). Similarly, a quantum algorithm $Q(|x\rangle)$ is said to be a quantum polynomial time (QPT) machine if it runs in time poly($|x|$) where $|x\rangle$ is a state in some Hilbert space $\mathcal{H}$. For any given circuit C, we denote $|C|$ to be the size of the circuit and $depth(C)$ to be its depth. We denote **BQP** to be the set of languages decidable by a QPT machine and **QMA** to be the set of all languages L where $L \in$ **QMA** if, for every $x \in L$, there exists a polynomial-sized quantum state (a witness) that makes a QPT machine accept x with high probability (and rejects otherwise). For any two distributions X and Y, we denote $X \overset{c}{\approx} Y$ whenever X and Y are computationally indistinguishable and $X \overset{s}{\approx} Y$ to mean that X and Y are statistically close. Throughout our work, we refer to the Quantum Random Oracle Model [20] (QROM) as an idealized model that allows adversaries to have black-box access to a quantum analogue of the classical random oracle that can input/output quantum states.

3.2 Time-Lock Puzzles

In this section, we restate the definition of a time-lock puzzle (TLP) [86].

Definition 3.1. A TLP scheme consists of three algorithms: TLP = (TLP.Setup, TLP.GenPuzzle, TLP.Solve) defined as follows:

- $\mathsf{TLP.Setup}(1^\lambda, \Delta) \to (tpk, tsk)$. A probabilistic algorithm that takes as input a security parameter, 1^λ, and time parameter Δ that specifies how long a message must remain hidden in seconds. It outputs a pair (tpk, tsk) that contains the scheme's public and private parameters, respectively.
- $\mathsf{TLP.GenPuzzle}(m, tpk, tsk) \to o$. A probabilistic algorithm that takes as input a solution m and (tpk, tsk). It outputs a puzzle o.
- $\mathsf{TLP.Solve}(tpk, o) \to s$. A deterministic algorithm that takes as input tpk and o. It outputs a solution s.

The following properties must also be satisfied:

- Completeness. For any message m it always holds that:
$$\mathsf{TLP.Solve}(tpk, \mathsf{TLP.GenPuzzle}(m, tpk, tsk)) = m$$
- Efficiency. The run-time of $\mathsf{TLP.Solve}(pk, o)$ is upper-bounded by $poly(\Delta, \lambda)$.

In certain schemes, such as [5], the secret key tsk may not be needed. The security of a TLP requires that the puzzle's solution remains confidential against all adversaries running in parallel within the time period, Δ. It also requires that an adversary cannot extract a solution in time $\delta(\Delta) < \Delta$, using a polynomial number of processors $poly'(\Delta)$ processors that run in parallel and after a large amount of pre-computation.

Definition 3.2. A TLP is secure if for all λ and Δ, all quantum polynomial time (QPT) adversaries $A := (A_1, A_2)$ where A_1 runs in total time $O(poly(\Delta, \lambda))$ and A_2 runs in parallel time $\delta(\Delta) < \Delta$ using at most $p(\Delta) = \text{poly}(\Delta)$ parallel processors, there is a negligible function $\text{negl}(\lambda)$, such that:

$$\Pr\left[\begin{array}{l}(tpk, tsk) \leftarrow \mathsf{TLP.Setup}(1^\lambda, \Delta)\\ (m_0, m_1, \text{state}) \leftarrow A_1(1^\lambda, tpk, \Delta)\\ b \xleftarrow{\$} \{0,1\}\\ o \leftarrow \mathsf{TLP.GenPuzzle}(m_b, tpk, tsk)\\ \hline b \leftarrow A_2(tpk, o, \text{state})\end{array}\right] \leq \frac{1}{2} + \text{negl}(\lambda)$$

In the literature of TLP, the security definition includes two adversaries A_1 and A_2 because they have different run-time constraints. In this work, we use the same approach.

3.2.1 Post Quantum Secure TLPs. Lai and Malavolta [62] proposed a candidate post-quantum secure sequential function relying on a lattice-based hash function [69]. Using the proposed sequential squaring, it is possible to easily construct a TLP. However, this TLP will require the puzzle generator to take as many steps as the puzzle solver. Very recently, Shweta et al. [5] proposed a TLP based on

lattices that can be instantiated using the SIS-sequential function of [62] (with judicious parameter settings). In this scheme, the puzzle generator can create puzzles more quickly than the solver can solve them. Our scheme can leverage any post-quantum secure TLP in a black-box manner.

Lemma 3.3 ([5]). *Assuming the quantum hardness of circular small-secret LWE [50], there exists a time-lock puzzle that is secure against polynomial-time quantum adversaries.*

3.3 Commitment Scheme

There are two parties involved in a commitment scheme, a *sender* and a *receiver*. The commitment scheme has two phases, *commit* and *open*. In the *commit* phase, the sender commits to a message m as $\mathtt{Com}(m, r) = com$, that involves a secret value, r. In the *open* phase, the sender sends the opening $\hat{m} := (m, r)$ to the receiver which verifies its correctness: $\mathtt{Ver}(com, \hat{m}) \stackrel{?}{=} 1$ and accepts if the output is 1. A commitment scheme must satisfy (a) *hiding*, it is infeasible for an adversary (i.e., the receiver) to learn any information about the committed message m, until the commitment com is opened, and (b) *binding*, it is infeasible for an adversary (i.e., the sender) to open a commitment com to different values $\hat{m}' := (m', r')$ than that was used in the commit phase, i.e., infeasible to find $\hat{m}'$, *s.t.* $\mathtt{Ver}(com, \hat{m}) = \mathtt{Ver}(com, \hat{m}') = 1$, where $\hat{m} \neq \hat{m}'$.

Definition B.1 in Appendix B provides a formal definition of a commitment scheme.

There exist efficient commitment schemes in the random oracle model using the well-known hash-based scheme such that committing is: $\mathtt{H}(x||r) = com$ and $\mathtt{Ver}(com, \hat{x})$ requires checking: $\mathtt{H}(x||r) \stackrel{?}{=} com$, where $\mathtt{H} : \{0,1\}^* \to \{0,1\}^\lambda$ is a collision-resistant hash function, i.e., the probability to find x and x' such that $\mathtt{H}(x) = \mathtt{H}(x')$ is negligible in the security parameter λ. There have also been various post-quantum commitment schemes [60, 91, 53, 89] by relying on LWE, learning parity with noise (LPN), or short integer solution (SIS) assumptions.

Lemma 3.4 ([89]). *Assuming the quantum hardness of the Short Integer Solutions (SIS) problem, there exists a computationally binding and statistically hiding commitment scheme.*

3.4 Quantum Verification Protocols

We recall the complexity class $\mathbf{QPIP}_\tau$ defined in [6] as the set of languages that have an interactive proof between a **BQP** prover and a hybrid classical-quantum verifier with a τ-qubit register. In our work, we will focus only on fully classical (**BPP**) verifiers (i.e. $\tau = 0$).

Definition 3.5. A $\mathbf{QPIP}_0$ protocol $\Pi = (P, V)$ for a **BQP** language L with completeness $c(.)$ and soundness $s(.)$ is an interactive protocol between a QPT prover P and a PPT verifier V denoted as $b \leftarrow \langle P, V(z_v)\rangle(1^\lambda, z)$ on shared input $z \in \{0,1\}^*$ where $b \in \{0,1\}$ is the output of V indicating acceptance/rejection and $z_v \in \{0,1\}^*$ is the verifier's private input. Also, the following properties hold:

- Completeness: For every $\lambda \in \mathbb{N}, z \in L$, and $z_v \in \{0,1\}^*$:

$$\Pr\left[\langle P, V(z_v)\rangle(z, 1^\lambda) = 1\right] \geq c(\lambda)$$

- Soundness: For any arbitrary QPT prover P^*, large enough $\lambda \in \mathbb{N}$, and all $z \notin L$ and $z_v \in \{0,1\}^*$:

$$\Pr\left[\langle P^*, V(z_v)\rangle(z, 1^\lambda) = 1\right] \leq s(\lambda)$$

where $c - s \geq 1/\mathrm{poly}(\lambda)$. Without loss of generality, we assume that $s = \mathrm{negl}(\lambda)$.

Given a $\mathbf{QPIP}_0$ protocol, one can use it to classically verify the computation of any **BQP** computation that was delegated to an untrusted quantum prover. This task is referred to in the literature as classical verification of quantum computation (CVQC). Simply put, one can define $L = \{(C, x, y) \mid C(x) = y\}$ to be the set of quantum circuit evaluations and their respective outputs. For simplicity, in this work, we assume that $y = 1$ for all circuit-input pairs (C, x) in L (i.e., circuits are Boolean) and restrict our attention to quantum circuits that accept and output classical strings. Since the classical verifier has no (efficient) way to evaluate this quantum circuit, it falls to the prover to provide a classical proof convincing the verifier of the evaluation's correctness.

In the original Mahadev CVQC protocol [67], the public parameter pk represents a sequence of $n = O(|C|)$ public keys $pk_1, \ldots, pk_n$ each used to evaluate a trapdoor claw-free function (TCF). These keys allow the prover to coherently evaluate the TCFs over its computation instance and send a classical string representing a commitment of its results. The verifier then initiates a 2-round challenge-response mechanism to test and verify the prover's claim.

As demonstrated in [7, 28], the above 4-round commit-and-measure protocol can be transformed into a non-interactive protocol (with setup) via the round-collapsing Fiat-Shamir transform [36], which was shown to be sound in the QROM [34, 64]. However, this transformation still results in a protocol that is privately verifiable. We describe their scheme as follows:

Protocol 3.6 (Privately Verifiable CVQC [28]). *Let $x \in L$ be the instance of a* **BQP** *language L. Assuming the hardness of LWE, there exists a construction of a 2-round privately verifiable CVQC protocol $\Pi = (V_1, P, V_{out})$ between verifier $V = (V_1, V_{out})$ and prover P in the QROM for language L that operates as follows:*

(1) $V_1(1^\lambda, x)$: Given security parameter 1^λ and an instance x, generate a public-private key pair (pk, sk), then send pk to the prover.
(2) $P(pk, x)$: Given x and pk, generate a classical proof string π and send it to V.
(3) $V_{out}(pk, sk, \pi)$: Given pk, the private verification key sk and proof π, verifier V either outputs 1 (accept) or 0 (reject).

Since the above protocol is a 2-round $\mathbf{QPIP}_0$, it has the same completeness and soundness properties formalized in Definition 3.5. CVQC schemes may also have the added property of being *publicly verifiable* where the correctness of the computation can be verified by any arbitrary third party (and not just by the delegator) using only the transcript of an interaction.

4 Time-Delayed Publicly Verifiable CVQC

We start by defining a publicly verifiable time-delayed CVQC scheme following the syntax of [80, 56] for classical verifiable computation schemes but with a notion that is closer to publicly verifiable non-adaptive preprocessing SNARGs [17]. Without loss of generality, we focus on the verifiability of Boolean circuits C since in order to

prove that $C'(x) = y$ for some $y \neq 1$, it suffices to instead prove that $C_y(x) = 1$ where C_y is the circuit that runs $C'(x)$ and outputs 1 if and only if $C'(x) = y$. It is thus natural to generalize this proof system to **QMA** languages of the form $L = \{(C, z) \mid \exists y : C(z) = y\}$.

Definition 4.1. A publicly verifiable time-delayed CVQC scheme for a family of Boolean quantum circuits $\{C_\lambda : \{0,1\}^{n(\lambda)} \rightarrow \{0,1\}\}_{n\in\mathbb{N}}$ with a time-bound $T = T(\lambda)$ consists of four algorithms $\mathsf{VC}_T = (\mathsf{VC.Setup}, \mathsf{VC.Prove}, \mathsf{VC.Reveal}, \mathsf{VC.Verify})$ that are defined as follows:

- $\mathsf{VC.Setup}(1^\lambda, C, x, T)$: A PPT algorithm that takes as input a security parameter 1^λ, a circuit $C \in C_\lambda$, input $x \in \{0,1\}^n$, and time bound T. It outputs a common reference string crs.
- $\mathsf{VC.Prove}(crs, C, x)$: A QPT algorithm, where given crs, a circuit $C \in C_\lambda$, and input $x \in \{0,1\}^n$, outputs a timestamped proof π_τ.
- $\mathsf{VC.Reveal}(crs)$: A QPT algorithm where, given crs outputs a string y that facilitates verification.
- $\mathsf{VC.Verify}(crs, C, x, \pi_\tau, y)$: A PPT algorithm where, given crs, a circuit $C \in C_\lambda$, input $x \in \{0,1\}^{\mathrm{poly}(\lambda)}$, a timestamped proof π_τ, and y, outputs 1 (accept) or 0 (reject).

The following properties must also be satisfied:

- Completeness: For every $\lambda \in \mathbb{N}, T = T(\lambda), C \in C_\lambda$, $x \in \{0,1\}^{\mathrm{poly}(\lambda)}$ such that $C(x) = 1$, the following must hold:

$$\Pr\left[\begin{array}{c|c} \mathsf{VC.Verify}(crs, C, & crs \leftarrow \mathsf{VC.Setup}(1^\lambda, C, x, T) \\ x, \pi_\tau, y) = 1 & \pi_\tau \leftarrow \mathsf{VC.Prove}(crs, C, x) \\ & y \leftarrow \mathsf{VC.Reveal}(crs) \end{array}\right] = 1$$

- Efficiency: For every λ, all algorithms VC.Setup runs in time $\mathrm{poly}(\lambda, |C|, |x|, T)$, while VC.Reveal, VC.Prove run in time that is $\mathrm{poly}(\lambda, |C|, |x|, T)$, and VC.Verify runs in time that is $\mathrm{poly}(\lambda, \log depth(C), |x|, \log T)$.
- Δ−Soundness: For every QPT adversary A that runs in time $\delta(\Delta) < \Delta$ for some $\Delta > T^{1+\epsilon} = \mathrm{poly}(T)$ and $\epsilon > 0$, there exists a negligible function $\mathrm{negl}(\cdot)$ such that:

$$\Pr[\mathbf{Exp}_A^{VC}[\lambda, T] = 1] \leq \mathrm{negl}(\lambda)$$

where $\mathbf{Exp}_A^{VC}[\lambda, T]$ is defined as the following experiment.

Experiment $\mathbf{Exp}_A^{VC}[\lambda, T]$:
$(C, x) \leftarrow A(1^\lambda)$ where $C \in C_\lambda$ and $x \in \{0,1\}^{\mathrm{poly}(\lambda)}$
$crs \leftarrow \mathsf{VC.Setup}(1^\lambda, C, x, T)$
$(\pi_\tau, y) \leftarrow A^{\mathcal{B}}(crs, C, x)$
$b_1 \leftarrow \mathsf{VC.Verify}(crs, C, x, \pi_\tau, y)$
$(sk, r) \leftarrow \mathsf{VC.Reveal}(crs)$
$b_2 \leftarrow \mathsf{VC.Verify}(crs, C, x, \pi_\tau, (sk, r))$
Output $(b_1 \vee b_2) \wedge (C(x) = 0 \vee sk \neq y)$

The experiment is essentially divided into two verifications that challenge the validity of the adversary's generated proof, one that is conducted before time Δ (where sk is hidden) and one that is conducted at time Δ after sk is revealed by VC.Reveal. The first pre-Δ verification tests the adversary's proof against its own guess of the secret key y. The second verification tests the *same* proof against the real secret key sk to ensure that a blind pre-Δ forgery of the proof still fails under sk. Only proofs that have been generated before sk is revealed are considered valid forgeries[1] and, in order to enforce this, algorithms are given access to a trusted "time-stamping" service (e.g., public blockchain) $\mathcal{B}(\cdot)$ [68] that can generate a verifiable time-stamp at time τ for any string. Thus, in order to win, the adversary must generate a false proof before time Δ that is valid against its own fake key y or against sk, when it is revealed.

4.1 The Proposed Scheme

Our construction, shown in Figure 2, makes use of Protocol 3.6, the 2-round CVQC protocol, along with a time-lock puzzle scheme to achieve public verifiability, and a commitment scheme to ensure the outsourced puzzle containing the verification key was correctly revealed. Figure 3 outlines the different phases of the protocol.

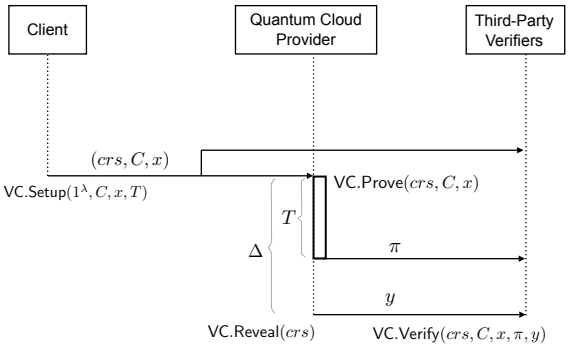


Figure 3: Outline of the post-quantum PV-CVQC protocol. The time to compute the delegated circuit is denoted by T. Thus the time to solve the puzzle should be set to some parameter $\Delta > T^{1+\epsilon}$ where $\epsilon > 0$.

The protocol starts by running the setup phase VC.Setup, which would be executed by an honest client who wants to outsource the computation of some circuit C on input x. The time bound $T = T(C_\lambda)$ is a function of the circuit family to which the computation belongs and can be set to be an upper bound on the depth of any $C \in C_\lambda$. The algorithm VC.Setup will then set the time-lock puzzle difficulty parameter Δ to be any value greater than $T + T_{prove}$ where T_{prove} is the running time of the prover's P algorithm. This ensures that the computation is guaranteed to be completed before the puzzle is solved. The verification algorithm of the privately verifiable algorithm V_1 is then run to obtain the instance-specific public-private parameter (pk, sk) pair of Π where the instance is a circuit-input pair (C, x). Finally, the puzzle and commitment of sk are generated and the CRS is constructed to contain the public parameter tpk for solving the puzzle, the public key pk for running the prover of Π, the puzzle o and commitment d of sk. Recall that the CRS is not sampled uniformly at random but is deliberately structured, which prevents us from simply replacing it with the output of a random oracle.

[1] In general, a privately verifiable scheme, which we make use of in our protocol, does not guarantee the soundness of proofs that have been generated *after* the secret verification key sk is disclosed.

Protocol 4.2 (Time-Delayed Publicly Verifiable CVQC). *Let $\Pi = (V_1, P, V_{out})$ be a 2-round privately verifiable CVQC in the QROM, let $(\mathsf{Com}, \mathsf{Ver})$ be a commitment scheme, and let $\mathsf{TLP} = (\mathsf{TLP.Setup}, \mathsf{TLP.GenPuzzle}, \mathsf{TLP.Solve})$ be a time-lock puzzle scheme. All algorithms have access to a timestamping service $\mathcal{B}$ (e.g., public blockchain); $\mathcal{B}$ does not escrow secrets. Verifiers may be offline at proof-generation time; they only need access to $\mathcal{B}$'s public record to validate the embedded timestamp later during verification. The construction of the time-delayed publicly verifiable CVQC $\mathsf{VC}_T = (\mathsf{VC.Setup}, \mathsf{VC.Prove}, \mathsf{VC.Reveal}, \mathsf{VC.Verify})$ for a family of quantum circuits C_λ proceeds as follows:*

- ***Setup Phase:*** $\mathsf{VC.Setup}(1^\lambda, C, x, T)$
  Given security parameter 1^λ, circuit $C \in C_\lambda$, input $x \in \{0,1\}^{\mathrm{poly}(\lambda)}$, and time-bound $T = T(\lambda)$:
  (1) Set $\Delta > T^{1+\epsilon}$ for some $\epsilon > 0$
  (2) Generate a public-private key pair for the time-lock puzzle $(tpk, tsk) \leftarrow \mathsf{TLP.Setup}(1^\lambda, \Delta)$
  (3) Run $(pk, sk) \leftarrow V_1(1^\lambda, (C, x))$
  (4) Let $d \leftarrow \mathsf{Com}(sk, r)$ where $r \xleftarrow{\$} \{0,1\}^\lambda$
  (5) Generate puzzle $o \leftarrow \mathsf{TLP.GenPuzzle}((sk, r), tpk, tsk)$
  (6) Output $crs = (tpk, pk, o, d, \Delta)$.
- ***Prove Phase:*** $\mathsf{VC.Prove}(crs, C, x)$
  Given crs, circuit $C \in C_\lambda$, and input $x \in \{0,1\}^{\mathrm{poly}(\lambda)}$:
  (1) Extract pk from crs
  (2) Run $\pi \leftarrow P(pk, (C, x))$
  (3) Timestamp proof: $\pi_\tau \leftarrow \mathcal{B}(\pi)$
  (4) Return π_τ
- ***Reveal Phase:*** $\mathsf{VC.Reveal}(crs)$
  Given crs:
  (1) Extract tpk, o from crs
  (2) Run $(sk', r') \leftarrow \mathsf{TLP.Solve}(tpk, o)$
  (3) Return $y = (sk', r')$
- ***Verify Phase:*** $\mathsf{VC.Verify}(crs, C, x, \pi_\tau, y)$:
  Given crs, circuit $C \in C_\lambda$, input $x \in \{0,1\}^{\mathrm{poly}(\lambda)}$, decommitment y, and timestamped proof string π_τ:
  (1) If $\tau > \Delta$, output 0
  (2) Extract (pk, d) from crs
  (3) Parse $y = (sk', r')$
  (4) Let $b \leftarrow \mathsf{Ver}(d, (sk', r'))$. If $b = 0$, output 0
  (5) Output $V_{out}(pk, sk', \pi_\tau)$

Figure 2: The Time-Delayed PV-CVQC Protocol in the QROM

The VC.Prove algorithm, typically executed by the server, is used to run the computation and proof generation procedure of Π using the public key pk and the statement (C, x). Concurrently, VC.Reveal is also executed, which starts solving the puzzle provided in the CRS. Since the puzzle has difficulty parameter $\Delta > T^{1+\epsilon}$, it is expected that VC.Prove would complete and output the proof π before VC.Reveal outputs the solution (the verification key).

Finally, when it comes to the verifier, it will run VC.Verify, which will execute the verification algorithm V_{out} of Π and output the result since it now has access to the key sk and the proof π.

Theorem 4.3. *If Com is a secure post-quantum commitment scheme, TLP is a secure post-quantum time-lock puzzle, and Π is a 2-round privately verifiable CVQC in the QROM, then Protocol 4.2 is a secure time-delayed publicly verifiable non-interactive CVQC protocol for a circuit class C under the quantum random oracle with CRS model, w.r.t. Definition 4.1.*

To prove the theorem we need to show that it satisfies the correctness, efficiency, and soundness properties of Definition 4. Completeness follows from the underlying 2-round CVQC protocol, the time-lock puzzle, and the commitment scheme. Specifically, for any $C \in C_\lambda$, $x \in \{0,1\}^n$ and $T = T(\lambda)$ and any honestly generated $crs = (tpk, pk, o, d, \Delta)$:

$$\mathsf{VC.Verify}(crs, C, x, \pi, y) = V_{out}(pk, sk, \pi) = 1$$

where $\pi \leftarrow P(pk, (C, x))$, $d \leftarrow \mathsf{Com}(sk, r)$, and $y = (sk, r) \leftarrow \mathsf{TLP.Solve}(tpk, o)$.

Regarding the efficiency requirement, it is clear that VC.Setup will run in time $\mathrm{poly}(\lambda, |C|, |x|, T)$ due to the size of pk. Furthermore, VC.Prove runs in time $\mathrm{poly}(\lambda, |C|, |x|, T)$ as it will need to compute the circuit and the proof, and VC.Reveal runs in time $\mathrm{poly}(\lambda, T)$ due to TLP.Solve, which will solve the puzzle in time $\Delta = \mathrm{poly}(T)$.

Remark 4.4 (Instance-Independent Variant). *In our construction, the CRS is circuit-dependent since the underlying 2-round CVQC requires this circuit to generate the verification key. Alternatively, Alagic et al. [7] show a variant of a 2-round privately verifiable CVQC*

that is instance-independent at the cost of increasing the number of parallel executions by a constant. This allows us to make VC.Setup *generate a CRS that is independent of the circuit-input pair* (C, x).

4.2 Proof of Security

We prove that the protocol has $\Delta-$soundness regarding Definition 4.2.

Proof. We show through a sequence of hybrids that the security of our scheme can be reduced to the soundness of the underlying 2-round CVQC, the security of the TLP, and the commitment scheme.

- H_0: This is the original experiment as stated in Definition 4.1 where the adversary's role as a malicious prover is to find a convincing false proof that passes verification.
- H_1: This is the same as H_0 except that Step 5 of the Verify phase is replaced with the following:

  $$\text{Output } V_{out}(pk, sk, \pi)$$

  In other words, the verification procedure of the underlying CVQC will use the original sk instead of sk'.
- H_2: This is the same as H_1 except that, in Step 5 of the Setup phase, the puzzle is generated using a random string pair $sk^* \xleftarrow{\$} \{0,1\}^n$, where $n = |sk| + |r|$, instead of (sk, r). That is, the puzzle is generated as follows:

  $$o^* \leftarrow \mathsf{TLP.GenPuzzle}(tpk, tsk, sk^*)$$

- H_3: The is the same as H_2 except that, in Step 4 of the Setup phase, the challenger commits to the same random string $sk \xleftarrow{\$} \{0,1\}^n$ used in the puzzle, instead of sk. That is, the commitment is generated as follows:

  $$d^* \leftarrow \mathsf{Commit}(sk^*)$$

Let $\mathrm{Adv}^{VC}_{H_i}(A) = |\Pr_{H_i}[\mathbf{Exp}^{VC}_{A}[\lambda, T] = 1]$ denote the adversary's success probability in the soundness game of Definition 4.1. The following sequence of claims will show that $\mathrm{Adv}^{VC}_{H_0}(A) \leq \mathrm{negl}(\lambda)$.

Claim 4.5. *If the commitment scheme is computationally binding against poly-time quantum adversaries, then:*

$$|Adv^{VC}_{H_0}(A) - Adv^{VC}_{H_1}(A)| \leq \mathrm{negl}(\lambda)$$

Proof. Let A be a QPT algorithm that succeeds in distinguishing between H_0 and H_1 with non-negligible advantage. Then there exists an efficient attacker B that acts as the challenger for A and breaks the binding property of the commitment scheme (as stated in Definition B.1) as follows. The attacker B would simulate the original game for A exactly up until Step 5 of the Verify phase. Here, H_0 and H_1 will differ only if $sk' \neq sk$ but $\mathsf{Ver}(d, (sk', r')) = 1$, which would imply breaking the binding property of the commitment scheme. Thus, as long as the commitment scheme is computationally binding the adversary's advantage of distinguishing between H_0 and H_1 is negligible. ∎

Claim 4.6. *Assuming that the TLP is secure against poly-time quantum adversaries, it holds then* $|Adv^{VC}_{H_1}(A) - Adv^{VC}_{H_2}(A)| \leq \mathrm{negl}(\lambda)$.

Proof. Let A be a QPT adversary that succeeds in distinguishing between H_1 and H_2 with non-negligible advantage. We show that there exists an efficient attacker $B = (B_1, B_2)$ that acts as the challenger for A and breaks the security of the underlying TLP scheme according to Definition 3.2 as follows. The attacker B_1 starts by running each step of $\mathsf{VC.Setup}(1^\lambda, (C, x))$, except in Step 2 it will first receive tpk from the TLP challenger, and in Step 5 it will ask for the challenge puzzle by submitting (m_0, m_1) as its chosen messages where $m_0 = (sk, r)$ contains the secret key from Step 3 and $m_1 = sk^*$ is some uniformly random string of size $|sk| + |r|$. Once it receives back the challenge puzzle $o_b \leftarrow \mathsf{TLP.GenPuzzle}(m_b, tpk, tsk)$, it runs A on $crs_b = (tpk, pk, o_b, d)$, which generates a proof π.

Similarly, B_2 will execute $A(crs_b, C, x)$. We note, since the running time of A is $\delta(\Delta) < \Delta$, B_2 will also have the same running time. Furthermore, if the TLP challenger chooses to generate a puzzle for $m_0 = (sk, r)$ then crs_b will have a puzzle of sk and the view of A will be that in H_1. On the other hand, if the TLP challenger chose to generate a puzzle for $m_1 = sk^*$ then the crs_b will have a puzzle of sk^* and the view of A will be that in H_2. The rest of the game remains unchanged between the two hybrids. Since B simulates A's challenge almost perfectly (with some negligible loss when $sk^* = (sk, r)$), the advantage of B is equal to that of A. Thus, as long as the TLP is secure, the adversary's advantage of distinguishing between H_1 and H_2 is negligible. ∎

Claim 4.7. *Assuming that the commitment scheme is statistically hiding against quantum adversaries, it holds that* $H_2 \overset{\$}{\approx} H_3$.

Proof. Let A be a QPT adversary that can distinguish between H_2 and H_3 with a non-negligible advantage. Then we can construct an adversary B that acts as the challenger for A and uses it to break the hiding property of the underlying commitment scheme. The attacker B starts by running each step in the Setup exactly as in H_2 except now in Step 4 it will first submit (m_0, m_1) to the challenger in the hiding game of the commitment scheme where $m_0 = (sk, r)$ and $m_1 = sk^*$ for some random $sk^* \xleftarrow{\$} \{0,1\}^{|sk|+|r|}$. Once it gets back the challenge commitment $d_b \leftarrow \mathsf{Com}(m_b)$, it runs A on $crs_b = (tpk, pk, o^*, d_b, \Delta)$ and (C, x). The rest of the steps remain the same. If the challenger chose to create a commitment for $m_0 = (sk, r)$ then the view of A will be that in H_2. If a commitment of $m_1 = sk^*$ was generated instead then the view of A will be that in H_3. Since B simulates A's challenge almost perfectly (with some negligible loss when $sk^* = (sk, r)$), the advantage of B in breaking the commitment scheme is equal to that of A. ∎

Claim 4.8. *If the 2-round privately verifiable CVQC scheme is computationally sound against all malicious QPT provers,* Com *is computationally binding/hiding, and the TLP is secure, then the advantage of A in* H_3 *is negligible.*

Proof. Let A be a QPT adversary that can break soundness in H_3. We then show how to construct an adversary B that will act as A's challenger and use it to break the soundness of the underlying privately verifiable CVQC scheme. B starts by requesting pk from its challenger (sk will be kept secret from B), creates a random puzzle o^* and commitment d^* of a random string, then sends $crs = (tpk, pk, o^*, d^*, \Delta)$ to A. Once A sends its proof π_τ and decommitment y, B will run $b_1 \leftarrow \mathsf{VC.Verify}(crs, C, x, \pi_\tau, y)$ and, after

it reveals (sk^*, r), also runs $b_2 \leftarrow \mathsf{VC.Verify}(crs, C, x, \pi_\tau, (sk^*, r))$. Since B exactly simulates the view of A in this game, we have that $\Pr[\text{B wins}] = \mathrm{Adv}^{VC}_{H_3}(A) \leq \Pr[b_1 \vee b_2 | C(x) = 0 \vee y \neq sk^*] \leq \Pr[b_1 | C(x) = 0 \vee y \neq sk^*] + \Pr[b_2 | C(x) = 0 \vee y \neq sk^*]$.

We note that if $b_1 = 1$ while $C(x) = 0$ and $y \neq sk^*$ then this implies that A found $sk' \neq sk^*$ which passes $\mathtt{Ver}(d^*, sk')$ or directly found sk^* from the commitment d^* or puzzle o^* (before time Δ). However, by the security of the underlying primitives, this only happens with negligible probability. Thus $\Pr[b_1 = 1 | C(x) = 0 \vee y \neq sk^*] \leq \mathrm{negl}(\lambda)$. Observe also that, during the second verification phase (post key release), if $b_2 = 1$ then it must be that $y = sk^*$ yet since $C(x) = 0$ this implies that π_τ has passed as a valid forged proof against the real secret key sk, violating the soundness of the underlying CVQC protocol. ■

Putting it all together to prove Theorem 4.3, we get that $H_0 \overset{\mathrm{c}}{\approx} H_1 \overset{\mathrm{c}}{\approx} H_2 \overset{\mathrm{s}}{\approx} H_3$, $\mathrm{Adv}^{H_3}_A \leq \mathrm{negl}(\lambda)$. Hence, the advantage of adversary A winning in H_0 is negligible. □

5 Experimental Results

In this section, we demonstrate the feasibility of our approach by simulating the protocol between a classical client who wishes to delegate quantum circuits of various sizes to a quantum server. Our implementation is a proof of concept with small security parameters, focusing primarily on measuring the time complexity of the parts of the protocol that dominate running time. In particular, we implemented the algorithms of the TLP (TLP.Setup, TLP.GenPuzzle, TLP.Solve), and the actual quantum circuit computation executed by the prover, and which happens to be part of the VC.Prove subroutine. Other parts of the protocol such commitments and VC.Verify were not simulated as they are relatively less time-consuming. The verifier's Feynman-Kitaev construction of the Hamiltonian [37] is assumed to be part of the preprocessing phase before the protocol starts so we do not consider it in our simulation.

Our implementation includes a variant of the TLP puzzle proposed in [5], which is based on reusable garbling circuit techniques. While the original scheme uses lattice-based succinct randomized encodings for their reusable garbling mechanism, we instead build upon the implementation of [47] which exhibits weaker security guarantees but is more practical. Additionally, relying on the fact that random oracles are unconditionally sequential [68], we instantiate our sequential function using SHA-256.

The simulations were performed on a t3.medium AWS instance with 2 vCPUs (3.1 GHz each), 4GB of memory, and access to an SV1 universal state vector simulator to execute quantum circuits (as the prover) on a cloud-based quantum simulation platform, specifically, AWS Braket. We use the Qiskit SDK [54] to define, build, transpile, and run the quantum circuits on the SV1 simulator. Each instance of the simulation is averaged over at least 20 different executions of the quantum circuit. The source code is available at [75].

5.1 Random Quantum Circuits

Table 2 shows the results of our experiments when tested against randomly generated quantum circuits of 5, 10, and 15 qubits, with a depth ranging between 10 and 300 gates. Random circuits were sampled using Qiskit's `random_circuit` function which generates circuits over a random distribution on the standard gates (e.g., Pauli, phase, swap, Hadamard, and controlled gates). For each qubit-depth setting, we simulated 20 different random quantum circuits and measured the time it takes for the prover to execute these circuits T. The client can use this to determine the required setting of Δ for any circuit of depth D such that the time to solve the puzzle is much greater than T. In all cases, we observed that, for the verifier, generating a puzzle is almost constant, whereas the bulk of the time was spent in the setup phase and is attributed to the reusable garbling mechanism (which is dependent on the circuit depth).

Across all settings, the prover's circuit execution time T increases with both depth and number of qubits (e.g. from 1.84 ms for 5 qubits at depth 10, up to 228.81 ms for 15 qubits at depth 300) which directly guides the client's choice of the time-delay parameter for releasing the verification parameters. In particular, the puzzle-solving TLP.Solve time is tuned to remain larger than the observed computation time. On the client side, puzzle generation TLP.GenPuzzle stays constant and negligible (≈ 0.05–0.07 ms). The dominant overhead appears in the puzzle setup cost TLP.Setup; however, this cost amortizes cleanly across multiple puzzle generations, as reflected by the amortized TLP.Setup column.

5.2 Enhanced Hybrid HHL

We also evaluated the protocol over a variant of the Harrow–Hassidim–Lloyd (HHL) circuits [46, 93], specifically enhanced hybrid HHL, which are known to take advantage of quantum speedup when solving large (sparse) linear systems and have been found to be particularly effective in finance applications including mean-variance portfolio optimization and risk assessment [84]. They are thus appropriate candidates for testing as classical algorithms would want to delegate these circuits to quantum-capable, yet potentially untrustworthy servers. We used the enhanced hybrid HHL algorithm (which has both classical and quantum computations) as opposed to the vanilla one since the former is more efficient in practice given the current quantum hardware capabilities.

Figure 4 shows the results for different sizes of circuits that correspond to different instances of the matrices which represent the system of linear equations. The depth can be used to estimate the time it takes to solve the circuit and thus determine the appropriate time Δ for which the puzzle should be solved. The parameter μ specifies the number of iterations of the sequential function to achieve this Δ. We observe from Figure 4(a) that the circuit depth grows approximately linearly with the size of the problem instance N. Figure 4(b) indicates that the measured execution time increases substantially with matrix (instance) size N, reaching on the order of minutes for the largest tested instances. These results allow the client to scale the sequential work factor μ of the time-lock puzzle so that puzzle solving completes (in Δ time) only after the prover finishes the HHL computation and produces the timestamped proof. Lastly, we see from Figure 4(c) that the average fidelity (which roughly represents the correctness of the quantum circuit upon measurement) of HHL degrades almost linearly with N. This is expected, as deeper circuits are prone to decoherence due to noise. While quantum error correction can be used to mitigate this issue, the depth (and hence the running time T) would significantly

Quantum Circuit			Time-Lock Puzzle (TLP)				
# Qubits	Depth	T (ms)	TLP.Solve (ms)	TLP.GenPuzzle (ms)	TLP.Setup (ms)	Amortized TLP.Setup (ms)	μ
5	10	1.84	13.63	0.05	1445.57	72.28	1
	20	2.31	13.63	0.05	1445.57	72.28	1
	50	4.56	13.63	0.05	1445.57	72.28	1
	100	8.13	13.63	0.05	1445.57	72.28	1
	200	16.84	32.97	0.05	6577.50	328.88	3
	300	22.84	45.83	0.06	10513.51	525.68	4
10	10	3.14	13.63	0.05	1445.57	72.28	1
	20	5.34	13.63	0.05	1445.57	72.28	1
	50	11.78	24.24	0.06	3672.94	183.65	2
	100	20.47	32.97	0.05	6577.50	328.88	3
	200	41.06	85.37	0.05	25915.06	1295.75	7
	300	58.01	117.25	0.05	48829.51	2441.48	10
15	10	10.52	24.24	0.06	3672.94	183.65	2
	20	19.39	32.97	0.05	6577.50	328.88	3
	50	49.51	103.16	0.05	40500.14	2025.01	9
	100	98.38	196.48	0.07	120891.41	6044.57	17
	200	168.56	339.30	0.05	52020.00	2601.00	30
	300	228.81	462.57	0.05	97161.80	4858.09	41

Table 2: The execution time (averaged over 20 trials) T of running a random quantum circuit for different numbers of qubits and circuit depths and the corresponding time elapsed for the time-lock puzzle setup (TLP.Setup, total and amortized over the 20 different puzzle generations), generation (TLP.GenPuzzle), and solving (TLP.Solve = Δ). The parameter μ corresponds to the number of executions of the sequential function to achieve the corresponding Δ time for any given circuit execution time T.

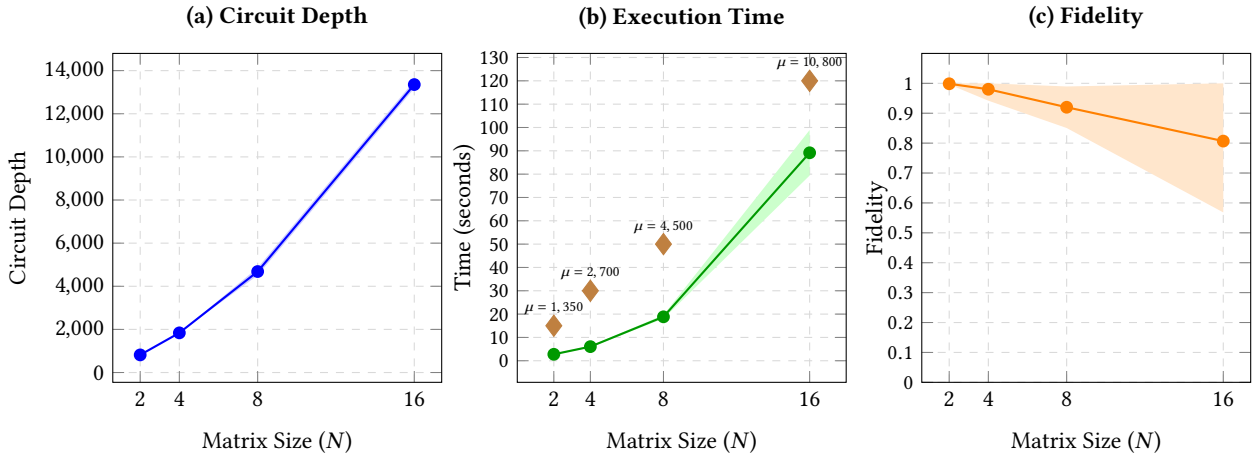


Figure 4: (a) Circuit depth scaling with matrix size, showing a linear relationship between the two. (b) The average running time of the HHL circuit for different matrix dimensions N and the corresponding μ for the time-lock puzzle that ensures it is solved only after the HHL circuit is complete. (c) The average fidelity (how well it adheres to the true solution) for each matrix size (N)

increase and the client should take into account this increase by setting μ appropriately and get the desired $\Delta > T$.

6 Conclusion and Future Work

This work proposes a verifiable computation scheme for quantum circuits which can be publicly verified by any third party after a set amount of time has elapsed. We demonstrate how a privately and classically verifiable quantum computation can be transformed into a publicly verifiable one by mainly relying on the recently introduced post-quantum time-lock puzzle [5]. Unlike previous work (that relied on iO), our scheme can be solely based on the LWE assumption, which is falsifiable and thus arguably more dependable than ones that are not falsifiable.

It remains open to find an approach that achieves (non-time-based) public verifiability in the plain model and under standard assumptions. It would also be useful to have a publicly verifiable

scheme that is applicable in the more common setting where the verification key is made available as soon as the circuit is delegated. Furthermore, the CRS in our scheme is not reusable (it can only be used to prove a single statement), regardless of whether the setup is instance-dependent or not. This is due to the fact that security (verifiability) is only guaranteed as long as the time-lock puzzle, which hides the secret verification key, is not solved. Therefore, it is an interesting open question to upgrade this scheme into one with a universal CRS that can be used to prove the correctness of multiple computations at once without increasing its size. Lastly, while verifiable computation schemes are generally concerned with malicious provers, extending security against malicious verifiers, who may attempt to falsely accuse the prover of providing an incorrect proof, is a promising future direction.

Acknowledgments

This work was funded by Kuwait Foundation for the Advancement of Sciences (KFAS) under project code: PA24-6TE-2493.

References

[1] Scott Aaronson, Alexandru Cojocaru, Alexandru Gheorghiu, and Elham Kashefi. 2019. Complexity-Theoretic Limitations on Blind Delegated Quantum Computation. en. *LIPIcs, Volume 132, ICALP 2019*, 132, 6:1–6:13. Artwork Size: 13 pages, 591258 bytes ISBN: 9783959771092 Medium: application/pdf Publisher: Schloss Dagstuhl – Leibniz-Zentrum für Informatik Version Number: 1.0. DOI: 10.4230/LIPICS.ICALP.2019.6.

[2] Aydin Abadi, Michele Ciampi, Aggelos Kiayias, and Vassilis Zikas. 2020. Timed signatures and zero-knowledge proofs - timestamping in the blockchain era -. In *ACNS* (Lecture Notes in Computer Science). Springer.

[3] Aydin Abadi and Aggelos Kiayias. 2021. Multi-instance publicly verifiable time-lock puzzle and its applications. In *FC*.

[4] Hamza Abusalah and Gennaro Avitabile. 2024. Black-Box Timed Commitments from Time-Lock Puzzles. Publication info: Published by the IACR in TCC 2024. (2024). Retrieved Nov. 25, 2024 from https://eprint.iacr.org/2024/1786.

[5] Shweta Agrawal, Giulio Malavolta, and Tianwei Zhang. 2024. Time-Lock Puzzles from Lattices. en. In *Advances in Cryptology – CRYPTO 2024*. Vol. 14922. Leonid Reyzin and Douglas Stebila, (Eds.) Series Title: Lecture Notes in Computer Science. Springer Nature Switzerland, Cham, 425–456. ISBN: 978-3-031-68382-4. DOI: 10.1007/978-3-031-68382-4_13.

[6] Dorit Aharonov, Michael Ben-Or, Elad Eban, and Urmila Mahadev. 2017. Interactive Proofs for Quantum Computations. arXiv:1704.04487 [quant-ph]. (Apr. 2017). DOI: 10.48550/arXiv.1704.04487.

[7] Gorjan Alagic, Andrew M. Childs, Alex B. Grilo, and Shih-Han Hung. 2020. Non-interactive Classical Verification of Quantum Computation. en. In *Theory of Cryptography*. Vol. 12552. Rafael Pass and Krzysztof Pietrzak, (Eds.) Series Title: Lecture Notes in Computer Science. Springer International Publishing, Cham, 153–180. DOI: 10.1007/978-3-030-64381-2_6.

[8] Arasu Arun, Joseph Bonneau, and Jeremy Clark. 2022. Short-lived Zero-Knowledge Proofs and Signatures. en. In *Advances in Cryptology – ASIACRYPT 2022*. Shweta Agrawal and Dongdai Lin, (Eds.) Springer Nature Switzerland, Cham, 487–516. ISBN: 978-3-031-22969-5. DOI: 10.1007/978-3-031-22969-5_17.

[9] James Bartusek, Andrea Coladangelo, Dakshita Khurana, and Fermi Ma. 2021. On the Round Complexity of Secure Quantum Computation. In *Advances in Cryptology – CRYPTO 2021: 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16–20, 2021, Proceedings, Part I*. Springer-Verlag, Berlin, Heidelberg, (Aug. 2021), 406–435. ISBN: 978-3-030-84241-3. DOI: 10.1007/978-3-030-84242-0_15.

[10] James Bartusek, Aparna Gupte, Saachi Mutreja, and Omri Shmueli. 2026. Classical Obfuscation of Quantum Circuits via Publicly-Verifiable QFHE. en. arXiv:2510.08400 [quant-ph]. (Jan. 2026). DOI: 10.48550/arXiv.2510.08400.

[11] James Bartusek, Yael Tauman Kalai, Alex Lombardi, Fermi Ma, Giulio Malavolta, Vinod Vaikuntanathan, Thomas Vidick, and Lisa Yang. 2022. Succinct Classical Verification of Quantum Computation. en. In *Advances in Cryptology – CRYPTO 2022*. Vol. 13508. Yevgeniy Dodis and Thomas Shrimpton, (Eds.) Series Title: Lecture Notes in Computer Science. Springer Nature Switzerland, Cham, 195–211. DOI: 10.1007/978-3-031-15979-4_7.

[12] James Bartusek, Fuyuki Kitagawa, Ryo Nishimaki, and Takashi Yamakawa. 2023. Obfuscation of Pseudo-Deterministic Quantum Circuits. en. In *Proceedings of the 55th Annual ACM Symposium on Theory of Computing*. ACM, Orlando FL USA, (June 2023), 1567–1578. ISBN: 978-1-4503-9913-5. DOI: 10.1145/3564246.3585179.

[13] James Bartusek and Giulio Malavolta. 2022. Indistinguishability Obfuscation of Null Quantum Circuits and Applications. en. *LIPIcs, Volume 215, ITCS 2022*, 215, 15:1–15:13. DOI: 10.4230/LIPICS.ITCS.2022.15.

[14] Stefanie Barz, Joseph F. Fitzsimons, Elham Kashefi, and Philip Walther. 2013. Experimental verification of quantum computation. en. *Nature Physics*, 9, 11, (Nov. 2013), 727–731. Publisher: Nature Publishing Group. DOI: 10.1038/nphys2763.

[15] Nir Bitansky, Ran Canetti, Alessandro Chiesa, Shafi Goldwasser, Huijia Lin, Aviad Rubinstein, and Eran Tromer. 2017. The Hunting of the SNARK. en. *Journal of Cryptology*, 30, 4, (Oct. 2017), 989–1066. DOI: 10.1007/s00145-016-9241-9.

[16] Nir Bitansky, Ran Canetti, Alessandro Chiesa, and Eran Tromer. 2013. Recursive composition and bootstrapping for SNARKS and proof-carrying data. en. In *Proceedings of the forty-fifth annual ACM symposium on Theory of Computing*. ACM, Palo Alto California USA, (June 2013), 111–120. ISBN: 978-1-4503-2029-0. DOI: 10.1145/2488608.2488623.

[17] Nir Bitansky, Alessandro Chiesa, Yuval Ishai, Omer Paneth, and Rafail Ostrovsky. 2013. Succinct Non-interactive Arguments via Linear Interactive Proofs. en. In *Theory of Cryptography*. Amit Sahai, (Ed.) Springer, Berlin, Heidelberg, 315–333. ISBN: 978-3-642-36594-2. DOI: 10.1007/978-3-642-36594-2_18.

[18] Dan Boneh, Joseph Bonneau, Benedikt Bünz, and Ben Fisch. [n. d.] Verifiable delay functions. In *CRYPTO'18*.

[19] Dan Boneh, Benedikt Bünz, and Ben Fisch. 2018. A survey of two verifiable delay functions. *IACR Cryptol. ePrint Arch.*

[20] Dan Boneh, Özgür Dagdelen, Marc Fischlin, Anja Lehmann, Christian Schaffner, and Mark Zhandry. 2010. Random Oracles in a Quantum World. Publication info: Published elsewhere. Unknown where it was published. (2010). Retrieved Apr. 4, 2025 from https://eprint.iacr.org/2010/428.

[21] Dan Boneh and Moni Naor. 2000. Timed commitments. In *CRYPTO*.

[22] Zvika Brakerski, Paul Christiano, Urmila Mahadev, Umesh Vazirani, and Thomas Vidick. 2018. A Cryptographic Test of Quantumness and Certifiable Randomness from a Single Quantum Device. In *2018 IEEE 59th Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE, Paris, (Oct. 2018), 320–331. ISBN: 978-1-5386-4230-6. DOI: 10.1109/FOCS.2018.00038.

[23] Zvika Brakerski, Venkata Koppula, Umesh Vazirani, and Thomas Vidick. 2020. Simpler Proofs of Quantumness. en. *LIPIcs, Volume 158, TQC 2020*, 158, 8:1–8:14. Artwork Size: 14 pages, 624972 bytes ISBN: 9783959771467 Medium: application/pdf Publisher: Schloss Dagstuhl – Leibniz-Zentrum für Informatik. DOI: 10.4230/LIPICS.TQC.2020.8.

[24] Zvika Brakerski and Gil Segev. 2018. Function-Private Functional Encryption in the Private-Key Setting. en. *Journal of Cryptology*, 31, 1, (Jan. 2018), 202–225. DOI: 10.1007/s00145-017-9255-y.

[25] Benedikt Bunz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. 2018. Bulletproofs: Short Proofs for Confidential Transactions and More. en. In *2018 IEEE Symposium on Security and Privacy (SP)*. IEEE, San Francisco, CA, (May 2018), 315–334. ISBN: 978-1-5386-4353-2. DOI: 10.1109/SP.2018.00020.

[26] Ran Canetti. 2001. Universally composable security: A new paradigm for cryptographic protocols. In *FOCS*. IEEE Computer Society.

[27] Ran Canetti, Yilei Chen, Justin Holmgren, Alex Lombardi, Guy N. Rothblum, Ron D. Rothblum, and Daniel Wichs. 2019. Fiat-Shamir: from practice to theory. en. In *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing*. ACM, Phoenix AZ USA, (June 2019), 1082–1090. ISBN: 978-1-4503-6705-9. DOI: 10.1145/3313276.3316380.

[28] Nai-Hui Chia, Kai-Min Chung, and Takashi Yamakawa. 2020. Classical Verification of Quantum Computations with Efficient Verifier. en. In *Theory of Cryptography*. Vol. 12552. Rafael Pass and Krzysztof Pietrzak, (Eds.) Series Title: Lecture Notes in Computer Science. Springer International Publishing, Cham, 181–206. DOI: 10.1007/978-3-030-64381-2_7.

[29] Arka Rai Choudhuri, Abhihsek Jain, and Zhengzhong Jin. 2022. SNARGs for \mathcalP from LWE. In *2021 IEEE 62nd Annual Symposium on Foundations of Computer Science (FOCS)*. ISSN: 2575-8454. (Feb. 2022), 68–79. DOI: 10.1109/FOCS52979.2021.00016.

[30] James CL Chow. 2024. Quantum computing in medicine. *Medical Sciences*.

[31] Kai-Min Chung, Yi Lee, Han-Hsuan Lin, and Xiaodi Wu. 2022. Constant-round blind classical verification of quantum sampling. In *EUROCRYPT* (Lecture Notes in Computer Science).

[32] Andrea Coladangelo, Alex B. Grilo, Stacey Jeffery, and Thomas Vidick. 2019. Verifier-on-a-Leash: New Schemes for Verifiable Delegated Quantum Computation, with Quasilinear Resources. en. In *Advances in Cryptology – EUROCRYPT 2019*. Yuval Ishai and Vincent Rijmen, (Eds.) Springer International Publishing, Cham, 247–277. ISBN: 978-3-030-17659-4. DOI: 10.1007/978-3-030-17659-4_9.

[33] Andrea Coladangelo, Thomas Vidick, and Tina Zhang. 2020. Non-interactive Zero-Knowledge Arguments for QMA, with Preprocessing. en. In *Advances in Cryptology – CRYPTO 2020*. Daniele Micciancio and Thomas Ristenpart, (Eds.)

Springer International Publishing, Cham, 799–828. ISBN: 978-3-030-56877-1. DOI: 10.1007/978-3-030-56877-1_28.

[34] Jelle Don, Serge Fehr, Christian Majenz, and Christian Schaffner. 2019. Security of the Fiat-Shamir Transformation in the Quantum Random-Oracle Model. en. In *Advances in Cryptology – CRYPTO 2019*. Alexandra Boldyreva and Daniele Micciancio, (Eds.) Springer International Publishing, Cham, 356–383. ISBN: 978-3-030-26951-7. DOI: 10.1007/978-3-030-26951-7_13.

[35] Thomas J. S. Durant, Elizabeth Knight, Brent G. Nelson, Sarah Dudgeon, Seung J. Lee, Dominic Walliman, Hobart Patrick Young, Lucila Ohno-Machado, and Wade L. Schulz. 2024. A primer for quantum computing and its applications to healthcare and biomedical research. *J. Am. Medical Informatics Assoc.*

[36] Amos Fiat and Adi Shamir. 1986. How To Prove Yourself: Practical Solutions to Identification and Signature Problems. en. In *Advances in Cryptology — CRYPTO' 86*. Andrew M. Odlyzko, (Ed.) Springer, Berlin, Heidelberg, 186–194. ISBN: 978-3-540-47721-1. DOI: 10.1007/3-540-47721-7_12.

[37] Joseph F. Fitzsimons and Michal Hajdušek. 2018. Post hoc verification of quantum computation. *Physical Review Letters*, 120, 4, (Jan. 2018), 040501. arXiv:1512.04375 [quant-ph]. DOI: 10.1103/PhysRevLett.120.040501.

[38] Joseph F. Fitzsimons and Elham Kashefi. 2017. Unconditionally verifiable blind quantum computation. (July 2017). DOI: 10.1103/PHYSREVA.96.012303.

[39] Craig Gentry and Daniel Wichs. 2011. Separating succinct non-interactive arguments from all falsifiable assumptions. en. In *Proceedings of the forty-third annual ACM symposium on Theory of computing*. ACM, San Jose California USA, (June 2011), 99–108. ISBN: 978-1-4503-0691-1. DOI: 10.1145/1993636.1993651.

[40] Alexandru Gheorghiu and Thomas Vidick. 2019. Computationally-Secure and Composable Remote State Preparation. en. In *2019 IEEE 60th Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE, Baltimore, MD, USA, (Nov. 2019), 1024–1033. ISBN: 978-1-7281-4952-3. DOI: 10.1109/FOCS.2019.00066.

[41] Riddhi Ghosal, Amit Sahai, and Brent Waters. 2023. Non-Interactive Publicly-Verifiable Delegation of Committed Programs. en. In *Public-Key Cryptography – PKC 2023*. Vol. 13941. Alexandra Boldyreva and Vladimir Kolesnikov, (Eds.) Series Title: Lecture Notes in Computer Science. Springer Nature Switzerland, Cham, 575–605. ISBN: 978-3-031-31370-7 978-3-031-31371-4. DOI: 10.1007/978-3-031-31371-4_20.

[42] Alex B. Grilo. 2019. A Simple Protocol for Verifiable Delegation of Quantum Computation in One Round. In *46th International Colloquium on Automata, Languages, and Programming (ICALP 2019)* (Leibniz International Proceedings in Informatics (LIPIcs)). Christel Baier, Ioannis Chatzigiannakis, Paola Flocchini, and Stefano Leonardi, (Eds.) Vol. 132. ISSN: 1868-8969. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 28:1–28:13. ISBN: 978-3-95977-109-2. DOI: 10.4230/LIPIcs.ICALP.2019.28.

[43] Jens Groth. 2010. Short Pairing-Based Non-interactive Zero-Knowledge Arguments. en. In *Advances in Cryptology - ASIACRYPT 2010*. Masayuki Abe, (Ed.) Springer, Berlin, Heidelberg, 321–340. ISBN: 978-3-642-17373-8. DOI: 10.1007/978-3-642-17373-8_19.

[44] Sam Gunn, Yael Tauman Kalai, Anand Natarajan, and Ági Villányi. 2025. Classical Commitments to Quantum States. In *Proceedings of the 57th Annual ACM Symposium on Theory of Computing* (STOC '25). Association for Computing Machinery, New York, NY, USA, (June 2025), 234–244. ISBN: 979-8-4007-1510-5. DOI: 10.1145/3717823.3718264.

[45] Aram W Harrow, Avinatan Hassidim, and Seth Lloyd. 2009. Quantum algorithm for linear systems of equations. *Physical review letters*, 103, 15, 150502.

[46] Aram W. Harrow, Avinatan Hassidim, and Seth Lloyd. 2009. Quantum algorithm for solving linear systems of equations. *Physical Review Letters*, 103, 15, (Oct. 2009), 150502. arXiv:0811.3171 [quant-ph]. DOI: 10.1103/PhysRevLett.103.150502.

[47] Christopher Harth-Kitzerow, Georg Carle, Fan Fei, Andre Luckow, and Johannes Klepsch. 2022. CRGC: A Practical Framework for Constructing Reusable Garbled Circuits: en. In *Proceedings of the 19th International Conference on Security and Cryptography*. SCITEPRESS - Science and Technology Publications, Lisbon, Portugal, 83–95. ISBN: 978-989-758-590-6. DOI: 10.5220/0011145300003283.

[48] Vojtech Havlicek, Antonio D. Corcoles, Kristan Temme, Aram W. Harrow, Abhinav Kandala, Jerry M. Chow, and Jay M. Gambetta. 2019. Supervised learning with quantum-enhanced feature spaces. *Nat.*

[49] Kentaro Honda. 2016. Publicly Verifiable Blind Quantum Computation. en. (Mar. 2016). Retrieved June 14, 2024 from http://arxiv.org/abs/1604.00116.

[50] Yao-Ching Hsieh, Huijia Lin, and Ji Luo. 2023. Attribute-Based Encryption for Circuits of Unbounded Depth from Lattices. English. In IEEE Computer Society, (Nov. 2023), 415–434. ISBN: 979-8-3503-1894-4. DOI: 10.1109/FOCS57990.2023.00031.

[51] Nouhaila Innan, Muhammad Al-Zafar Khan, and Mohamed Bennai. 2023. Financial fraud detection: A comparative study of quantum machine learning models. *CoRR*.

[52] Abhishek Jain and Zhengzhong Jin. 2022. Indistinguishability Obfuscation via Mathematical Proofs of Equivalence. In *2022 IEEE 63rd Annual Symposium on Foundations of Computer Science (FOCS)*. ISSN: 2575-8454. (Oct. 2022), 1023–1034. DOI: 10.1109/FOCS54457.2022.00100.

[53] Abhishek Jain, Stephan Krenn, Krzysztof Pietrzak, and Aris Tentes. 2012. Commitments and efficient zero-knowledge proofs from learning parity with noise. In *ASIACRYPT*.

[54] Ali Javadi-Abhari et al. 2024. Quantum computing with Qiskit. (2024). arXiv: 2405.08810 [quant-ph]. DOI: 10.48550/arXiv.2405.08810.

[55] Zhengfeng Ji. 2016. Classical verification of quantum proofs. In *Proceedings of the forty-eighth annual ACM symposium on Theory of Computing* (STOC '16). Association for Computing Machinery, New York, NY, USA, (June 2016), 885–898. ISBN: 978-1-4503-4132-5. DOI: 10.1145/2897518.2897634.

[56] Yael Tauman Kalai, Omer Paneth, and Lisa Yang. 2019. How to delegate computations publicly. en. In *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing*. ACM, Phoenix AZ USA, (June 2019), 1115–1124. ISBN: 978-1-4503-6705-9. DOI: 10.1145/3313276.3316411.

[57] Sai Krishna Kandula, Nagaveni Katam, Pranav Reddy Kangari, Adithya Hijmal, Rakesh Gurrala, and Mohammed Mahmoud. 2023. Quantum computing potentials for drug discovery. In *International Conference on Computational Science and Computational Intelligence (CSCI)*. IEEE.

[58] Elham Kashefi, Dominik Leichtle, Luka Music, and Harold Ollivier. 2024. Verification of Quantum Computations without Trusted Preparations or Measurements. en. arXiv:2403.10464 [quant-ph]. (Mar. 2024). Retrieved Sept. 2, 2024 from http://arxiv.org/abs/2403.10464.

[59] Jonathan Katz, Julian Loss, and Jiayu Xu. 2020. On the Security of Time-Lock Puzzles and Timed Commitments. en. In *Theory of Cryptography*. Rafael Pass and Krzysztof Pietrzak, (Eds.) Springer International Publishing, Cham, 390–413. ISBN: 978-3-030-64381-2. DOI: 10.1007/978-3-030-64381-2_14.

[60] Akinori Kawachi, Keisuke Tanaka, and Keita Xagawa. 2008. Concurrently Secure Identification Schemes Based on the Worst-Case Hardness of Lattice Problems. en. In *Advances in Cryptology - ASIACRYPT 2008*. Josef Pieprzyk, (Ed.) Springer, Berlin, Heidelberg, 372–389. ISBN: 978-3-540-89255-7. DOI: 10.1007/978-3-540-89255-7_23.

[61] Joe Kilian. 1992. A note on efficient zero-knowledge proofs and arguments (extended abstract). en. In *Proceedings of the twenty-fourth annual ACM symposium on Theory of computing - STOC '92*. ACM Press, Victoria, British Columbia, Canada, 723–732. ISBN: 978-0-89791-511-3. DOI: 10.1145/129712.129782.

[62] Russell W. F. Lai and Giulio Malavolta. 2023. Lattice-Based Timed Cryptography. en. In *Advances in Cryptology – CRYPTO 2023*. Vol. 14085. Helena Handschuh and Anna Lysyanskaya, (Eds.) Series Title: Lecture Notes in Computer Science. Springer Nature Switzerland, Cham, 782–804. DOI: 10.1007/978-3-031-38554-4_25.

[63] Yehuda Lindell. 2015. An Efficient Transform from Sigma Protocols to NIZK with a CRS and Non-programmable Random Oracle. en. In *Theory of Cryptography*. Yevgeniy Dodis and Jesper Buus Nielsen, (Eds.) Springer, Berlin, Heidelberg, 93–109. ISBN: 978-3-662-46494-6. DOI: 10.1007/978-3-662-46494-6_5.

[64] Qipeng Liu and Mark Zhandry. 2019. Revisiting Post-quantum Fiat-Shamir. en. In *Advances in Cryptology – CRYPTO 2019*. Alexandra Boldyreva and Daniele Micciancio, (Eds.) Springer International Publishing, Cham, 326–355. ISBN: 978-3-030-26951-7. DOI: 10.1007/978-3-030-26951-7_12.

[65] Wen-Jie Liu, Zi-Xian Li, Wen-Bo Li, and Qi Yang. 2023. Public verifiable measurement-only blind quantum computation based on entanglement witnesses. en. *Quantum Information Processing*, 22, 3, (Mar. 2023), 137. arXiv:2310.02922 [quant-ph]. DOI: 10.1007/s11128-023-03859-9.

[66] Yi Liu, Qi Wang, and Siu-Ming Yiu. 2022. Towards practical homomorphic time-lock puzzles: applicability and verifiability. In *ESORICS*.

[67] Urmila Mahadev. 2018. Classical Verification of Quantum Computations. en. In *2018 IEEE 59th Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE, Paris, (Oct. 2018), 259–267. ISBN: 978-1-5386-4230-6. DOI: 10.1109/FOCS.2018.00033.

[68] Mohammad Mahmoody, Tal Moran, and Salil Vadhan. 2013. Publicly verifiable proofs of sequential work. In *Proceedings of the 4th conference on Innovations in Theoretical Computer Science* (ITCS '13). Association for Computing Machinery, New York, NY, USA, (Jan. 2013), 373–388. ISBN: 978-1-4503-1859-4. DOI: 10.1145/2422436.2422479.

[69] Mohammad Mahmoody, Tal Moran, and Salil Vadhan. 2011. Time-Lock Puzzles in the Random Oracle Model. en. In *Advances in Cryptology – CRYPTO 2011*. Vol. 6841. David Hutchison et al., (Eds.) Series Title: Lecture Notes in Computer Science. Springer Berlin Heidelberg, Berlin, Heidelberg, 39–50. ISBN: 978-3-642-22791-2 978-3-642-22792-9. DOI: 10.1007/978-3-642-22792-9_3.

[70] Giulio Malavolta and Sri Aravinda Krishnan Thyagarajan. 2019. Homomorphic time-lock puzzles and applications. In *CRYPTO*.

[71] Walid El Maouaki, Taoufik Said, and Mohamed Bennai. 2024. Quantum support vector machine for prostate cancer detection: a performance analysis. *arXiv preprint arXiv:2403.07856*.

[72] Timothy C May. 1993. Timed-release crypto. (1993). https://cypherpunks.venona.com/date/1993/02/msg00129.html.

[73] Tony Metger, Anand Natarajan, and Tina Zhang. 2024. Succinct Arguments for QMA from Standard Assumptions via Compiled Nonlocal Games. In *2024*

IEEE 65th Annual Symposium on Foundations of Computer Science (FOCS). ISSN: 2575-8454. (Oct. 2024), 1193–1201. DOI: 10.1109/FOCS61266.2024.00078.

[74] Silvio Micali. 1994. A SECURE AND EFFICIENT DIGITAL SIGNATURE ALGORITHM. en.

[75] Ameer Mohammed, Aydin Abadi, and Jaffer Mahdi. 2026. Source code for time-delayed publicly verifiable quantum computation with classical verifiers. https://github.com/asadeq/timed-quantum-verification. (2026).

[76] Tomoyuki Morimae and Joseph F. Fitzsimons. 2018. Post hoc verification with a single prover. *Physical Review Letters*, 120, 4, (Jan. 2018), 040501. arXiv:1603.06046 [quant-ph]. DOI: 10.1103/PhysRevLett.120.040501.

[77] Tomoyuki Morimae, Daniel Nagaj, and Norbert Schuch. 2016. Quantum proofs can be verified using only single-qubit measurements. *Physical Review A*, 93, 2, (Feb. 2016), 022326. Publisher: American Physical Society. DOI: 10.1103/PhysRevA.93.022326.

[78] Tomoyuki Morimae and Takashi Yamakawa. 2022. Classically Verifiable NIZK for QMA with Preprocessing. en. In *Advances in Cryptology – ASIACRYPT 2022*. Shweta Agrawal and Dongdai Lin, (Eds.) Springer Nature Switzerland, Cham, 599–627. ISBN: 978-3-031-22972-5. DOI: 10.1007/978-3-031-22972-5_21.

[79] Moni Naor. 2003. On Cryptographic Assumptions and Challenges. en. In *Advances in Cryptology - CRYPTO 2003*. Dan Boneh, (Ed.) Springer, Berlin, Heidelberg, 96–109. ISBN: 978-3-540-45146-4. DOI: 10.1007/978-3-540-45146-4_6.

[80] Bryan Parno, Mariana Raykova, and Vinod Vaikuntanathan. 2012. How to Delegate and Verify in Public: Verifiable Computation from Attribute-Based Encryption. en. In *Theory of Cryptography*. Vol. 7194. David Hutchison et al., (Eds.) Series Title: Lecture Notes in Computer Science. Springer Berlin Heidelberg, Berlin, Heidelberg, 422–439. ISBN: 978-3-642-28914-9. DOI: 10.1007/978-3-642-28914-9_24.

[81] Chris Peikert and Sina Shiehian. 2019. Noninteractive Zero Knowledge for NP from (Plain) Learning with Errors. en. In *Advances in Cryptology – CRYPTO 2019*. Vol. 11692. Alexandra Boldyreva and Daniele Micciancio, (Eds.) Series Title: Lecture Notes in Computer Science. Springer International Publishing, Cham, 89–114. ISBN: 978-3-030-26948-7. DOI: 10.1007/978-3-030-26948-7_4.

[82] Krzysztof Pietrzak. 2019. Simple verifiable delay functions. In *10th Innovations in Theoretical Computer Science Conference, ITCS 2019, January 10-12, 2019, San Diego, California, USA*. Schloss Dagstuhl - Leibniz-Zentrum für Informatik.

[83] Raihan Ur Rasool, Hafiz Farooq Ahmad, Wajid Rafique, Adnan Qayyum, Junaid Qadir, and Zahid Anwar. 2023. Quantum computing for healthcare: A review. *Future Internet*.

[84] Patrick Rebentrost, Brajesh Gupt, and Thomas R. Bromley. 2018. Quantum computational finance: Monte Carlo pricing of financial derivatives. *Physical Review A*, 98, 2, (Aug. 2018), 022321. arXiv:1805.00109 [quant-ph]. DOI: 10.1103/PhysRevA.98.022321.

[85] Oded Regev. 2005. On lattices, learning with errors, random linear codes, and cryptography. en. In *Proceedings of the thirty-seventh annual ACM symposium on Theory of computing*. ACM, Baltimore MD USA, (May 2005), 84–93. ISBN: 978-1-58113-960-0. DOI: 10.1145/1060590.1060603.

[86] R. L. Rivest, A. Shamir, and D. A. Wagner. 1996. Time-lock Puzzles and Timed-release Crypto. Tech. rep.

[87] Omri Shmueli. 2021. Multi-theorem Designated-Verifier NIZK for QMA. en. In *Advances in Cryptology – CRYPTO 2021*. Tal Malkin and Chris Peikert, (Eds.) Springer International Publishing, Cham, 375–405. ISBN: 978-3-030-84242-0. DOI: 10.1007/978-3-030-84242-0_14.

[88] Yuki Takeuchi and Tomoyuki Morimae. 2018. Verification of Many-Qubit States. *Physical Review X*, 8, 2, (June 2018), 021060. Publisher: American Physical Society. DOI: 10.1103/PhysRevX.8.021060.

[89] Hoeteck Wee and David J. Wu. 2023. Succinct Vector, Polynomial, and Functional Commitments from Lattices. en. In *Advances in Cryptology – EUROCRYPT 2023*. Vol. 14006. Carmit Hazay and Martijn Stam, (Eds.) Series Title: Lecture Notes in Computer Science. Springer Nature Switzerland, Cham, 385–416. ISBN: 978-3-031-30619-8 978-3-031-30620-4. DOI: 10.1007/978-3-031-30620-4_13.

[90] Benjamin Wesolowski. 2019. Efficient verifiable delay functions. In *EUROCRYPT*.

[91] Xiang Xie, Rui Xue, and Minqian Wang. 2013. Zero knowledge proofs from ring-lwe. In *CANS*.

[92] Romina Yalovetzky, Pierre Minssen, Dylan Herman, and Marco Pistoia. 2024. Solving linear systems on quantum hardware with hybrid hhl++. *Scientific Reports*.

[93] Romina Yalovetzky, Pierre Minssen, Dylan Herman, and Marco Pistoia. 2024. Solving linear systems on quantum hardware with hybrid HHL++. en. *Scientific Reports*, 14, 1, (Sept. 2024), 20610. DOI: 10.1038/s41598-024-69077-0.

[94] Jiayu Zhang. 2022. Classical Verification of Quantum Computations in Linear Time. en. In *2022 IEEE 63rd Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE, Denver, CO, USA, (Oct. 2022), 46–57. ISBN: 978-1-66545-519-0. DOI: 10.1109/FOCS54457.2022.00012.

A Open Science Appendix

The code used containing the implementation of the protocol used to obtain our experimental results and instructions to run it can be found at [75].

B Formal Definition of Commitment Scheme

In this section, we present formal definition of a commitment scheme.

Definition B.1. A commitment scheme consists of two PPT algorithms $(\mathtt{Com}, \mathtt{Ver})$ with the following properties:

- Perfect completeness: For any m and r:
$$\mathtt{Ver}(\mathtt{Com}(m, r), (m, r)) = 1$$
- Computationally binding: For any QPT adversary A that outputs (m_0, r_0, m_1, r_1, c) such that $m_0 \neq m_1$ and $|m_0| = |m_1|$ the following holds for sufficiently large λ:
$$\Pr\left[\ \forall b \in \{0,1\}, \mathtt{Ver}(c, (m_b, r_b)) = 1\ \right] \leq \mathrm{negl}(\lambda)$$
- Statistically hiding: For any (potentially unbounded) adversary A the following holds for sufficiently large λ:
$$\Pr\left[\begin{array}{c} m_0 \neq m_1 \\ |m_0| = |m_1| \\ b' = b \end{array} \middle| \begin{array}{c} (m_0, m_1, st) \leftarrow A(1^\lambda) \\ b \xleftarrow{\$} \{0,1\} \\ c \leftarrow \mathtt{Com}(m_b, r) \\ b' \leftarrow A(st, c) \end{array}\right] \leq \frac{1}{2} + \mathrm{negl}(\lambda)$$